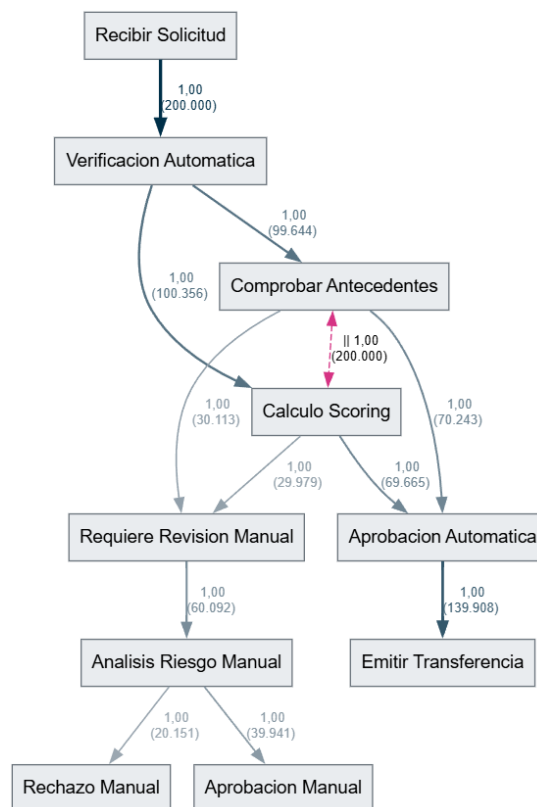


Process Miner

Desarrollo de una herramienta para el descubrimiento y análisis de procesos mediante Minería de Procesos



Alejandro Lara Lara

Índice

1. Introducción	3
2. Conceptos fundamentales	4
2.1. Event Log	4
2.2. Trazas	5
3. Arquitectura del sistema	5
4. Parseo y procesamiento inicial	6
4.1. Lectura del registro	6
5. Optimización del parseo	7
6. Descubrimiento del proceso mediante Alpha Miner	8
7. Tratamiento del ruido	9
8. Performance Mining	10
9. Resource Mining	11
10.Heuristic Miner	12
10.1. Umbrales y filtrado del ruido	13
11.Análisis de variantes	15
12.API Web	16
13.Arquitectura Stateless	17
14.Frontend	17
15.Soporte de formatos	19
16.Pruebas de rendimiento y escalabilidad	19
17.Despliegue	20
18.Flujo completo de funcionamiento	21
18.1. Carga del registro	21
18.2. Envío a la API	21
18.3. Parseo	22
18.4. Construcción de las trazas	22

18.5. Descubrimiento del proceso	22
18.6. Análisis adicional	22
18.7. Generación de resultados	22
18.8. Visualización	22
18.9. Exportación	22
19.Resultados obtenidos	23
20.Conclusiones	24

1. Introducción

En numerosas organizaciones, las actividades que forman parte de un proceso de negocio quedan registradas automáticamente en los sistemas informáticos que las gestionan. Cada vez que se realiza una acción, el sistema puede almacenar información como el identificador del caso al que pertenece, la actividad realizada, la fecha y hora en la que tuvo lugar, el recurso que la ejecutó o el coste asociado.

El conjunto de estos registros recibe el nombre de *event log* o registro de eventos. A partir de estos datos es posible estudiar cómo se ejecutan realmente los procesos, detectar los caminos más habituales, localizar cuellos de botella, analizar costes y tiempos de ejecución y estudiar la participación de los distintos recursos.

Sin embargo, un *event log* puede contener cientos de miles o incluso millones de eventos, por lo que analizarlo manualmente resulta impracticable. Además, el orden en el que aparecen las actividades no tiene por qué corresponder con un modelo de proceso previamente definido. Es necesario, por tanto, disponer de herramientas capaces de transformar estos registros en información estructurada y comprensible.

La **Minería de Procesos** (*Process Mining*) surge precisamente para resolver este problema. Su objetivo es extraer conocimiento sobre los procesos a partir de los registros de eventos generados durante su ejecución.

En términos generales, el procesamiento realizado por una herramienta de Minería de Procesos puede dividirse en varias etapas:

1. Adquisición de los datos mediante un registro de eventos.
2. Parseo y estructuración de los datos.
3. Agrupación de los eventos en trazas.
4. Descubrimiento del proceso.
5. Análisis del rendimiento.
6. Análisis de los recursos.
7. Análisis de las variantes del proceso.
8. Visualización de los resultados.

El objetivo de este proyecto es desarrollar una herramienta capaz de realizar estas operaciones sobre grandes registros de eventos, prestando especial atención a la eficiencia del procesamiento y a la facilidad de interpretación de los resultados.

El sistema se ha desarrollado mediante una arquitectura formada por un núcleo de procesamiento en C#, una API web basada en ASP.NET Core y una interfaz web desarrollada

con React y TailwindCSS.

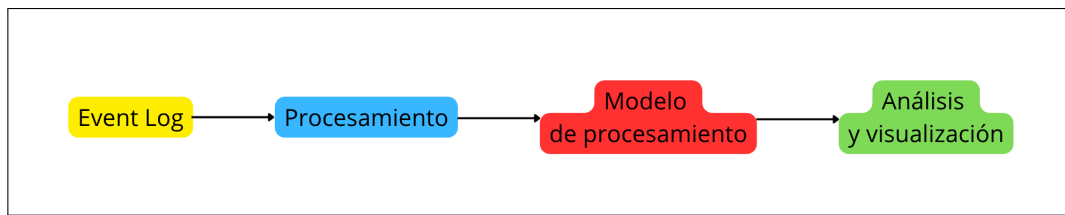


Figura 1: Flujo general de la Minería de Procesos.

2. Conceptos fundamentales

2.1. Event Log

El elemento fundamental sobre el que trabaja el sistema es el *registro de eventos* o *event log*.

Cada fila del registro representa un evento producido durante la ejecución de un proceso. Entre los datos utilizados por la aplicación se encuentran:

- Identificador del caso o proceso.
- Nombre de la actividad.
- Marca temporal.
- Recurso que ejecuta la actividad.
- Coste asociado a la actividad.

Por ejemplo, un proceso relacionado con una solicitud bancaria podría estar formado por actividades como:

- Recibir solicitud.
- Verificación automática.
- Comprobar antecedentes.
- Cálculo de *scoring*.
- Aprobación automática.
- Emisión de transferencia.
- Revisión manual.
- Análisis de riesgo.
- Rechazo.

Un único proceso puede contener varias de estas actividades y cada proceso puede seguir una secuencia diferente.

2.2. Trazas

Una **traza** representa la secuencia de eventos correspondiente a un caso concreto.

Por ejemplo:

Recibir Solicitud → Verificación Automática → Aprobación Automática → Emitir Transferencia

representa una posible ejecución del proceso.

Para poder analizar correctamente una traza, sus eventos deben estar ordenados cronológicamente. El sistema agrupa inicialmente los eventos mediante su identificador de caso y posteriormente los ordena temporalmente.

La ordenación de los eventos presenta una complejidad de:

$$O(n \log n)$$

donde n representa el número de eventos pertenecientes al conjunto que se está ordenando.

Case_ID	Timestamp	Activity	Resource	Cost
2001	2026-08-01 08:00:00	Recibir_Solicitud	Sistema	0
2001	2026-08-01 08:05:00	Verificacion_Automatica	API	2
2001	2026-08-01 08:10:00	Comprobar_Antecedentes	API_Legal	1
2001	2026-08-01 08:12:00	Calculo_Scoring	API_Riesgos	1
2001	2026-08-01 08:15:00	Aprobacion_Automatica	Sistema	0
2001	2026-08-01 08:20:00	Emitir_Transferencia	Banco	5
2002	2026-08-01 09:00:00	Recibir_Solicitud	Sistema	0
2002	2026-08-01 09:05:00	Verificacion_Automatica	API	2

Figura 2: Ejemplo de un registro de eventos utilizado como entrada.

3. Arquitectura del sistema

El proyecto se ha dividido en diferentes componentes con responsabilidades independientes.

La estructura inicial del proyecto está formada por:

- **ProcessMiner.Core**: núcleo de procesamiento y algoritmos de Minería de Procesos.
- **ProcessMiner.ConsoleClient**: aplicación utilizada durante las primeras fases de desarrollo y pruebas.
- **ProcessMiner.Tests**: entorno destinado a las pruebas del sistema.

Posteriormente se incorporó una API web desarrollada con **ASP.NET Core**, que utiliza el núcleo del proyecto para procesar los archivos y devolver los resultados al cliente web.

La interfaz de usuario se desarrolló con **React** y **TailwindCSS**, permitiendo cargar los registros y consultar los diferentes resultados del análisis.

La arquitectura final puede resumirse mediante el siguiente flujo:

Usuario → React → ASP.NET Core API → ProcessMiner.Core → Resultados → React

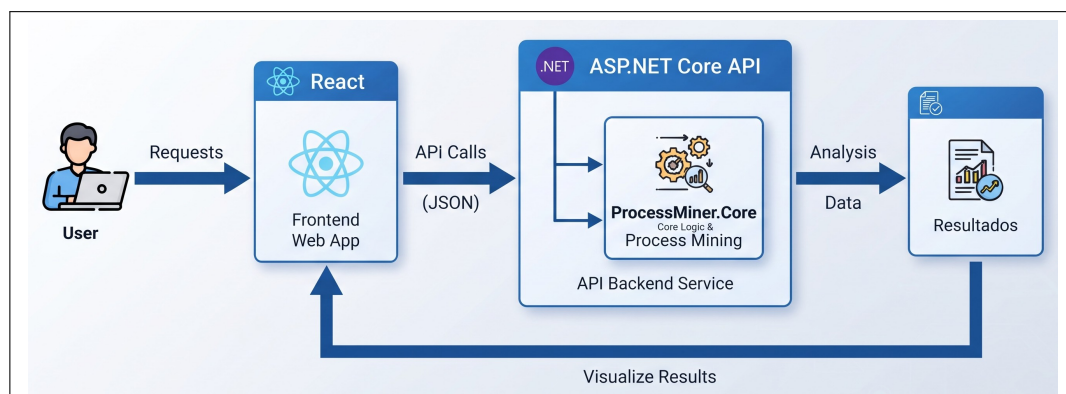


Figura 3: Arquitectura general de Process Miner.

4. Parseo y procesamiento inicial

4.1. Lectura del registro

La primera etapa del proyecto consistió en desarrollar el sistema capaz de leer un registro de eventos y convertir sus filas en objetos internos.

Para ello se crearon las clases `LogEvent` y `Trace`.

`LogEvent` representa un evento individual, mientras que `Trace` representa el conjunto ordenado de eventos pertenecientes a un mismo caso.

Una vez leído el archivo, los eventos se agrupan por identificador de proceso y se ordenan temporalmente.

Esta estructura permite desacoplar el formato original del archivo de los algoritmos de Minería de Procesos, ya que todos ellos trabajan sobre las estructuras internas creadas durante esta fase.

5. Optimización del parseo

Dado que uno de los objetivos del proyecto es trabajar con registros de gran tamaño, el rendimiento del parseo se convirtió en una parte importante del desarrollo.

La primera implementación utilizaba la función `Split` para separar los campos de cada línea. Aunque esta solución resulta sencilla, genera objetos temporales en memoria.

Para reducir la presión sobre el recolector de basura de .NET se desarrolló un parser alternativo basado en `ReadOnlySpan<char>`.

Esta técnica permite trabajar directamente sobre fragmentos de la cadena original sin generar necesariamente nuevos objetos para cada fragmento.

En una prueba con 200 000 procesos y una media de cinco eventos por proceso, el tiempo de ejecución pasó de aproximadamente:

2266 ms

a:

2125 ms

Además de la reducción del tiempo, el principal beneficio obtenido fue la reducción de la presión sobre la memoria.

Posteriormente se estudiaron diferentes estrategias mediante benchmarks. Se comprobó que una optimización aplicada únicamente al parseo no necesariamente proporciona una mejora en el procesamiento completo.

Por este motivo se incorporó una caché para evitar la creación repetida de determinadas cadenas.

Aunque esta caché incrementaba el tiempo empleado exclusivamente en el parseo, produjo una mejora significativa en el *pipeline* completo, debido a que se reutilizan referencias en lugar de generar continuamente nuevas cadenas.

En los benchmarks realizados, el uso de la caché produjo mejoras en el *pipeline* completo de entre aproximadamente un 17 % y un 27 % en los casos de mayor tamaño.

Finalmente se seleccionó para producción el **Fast Parser con caché**.

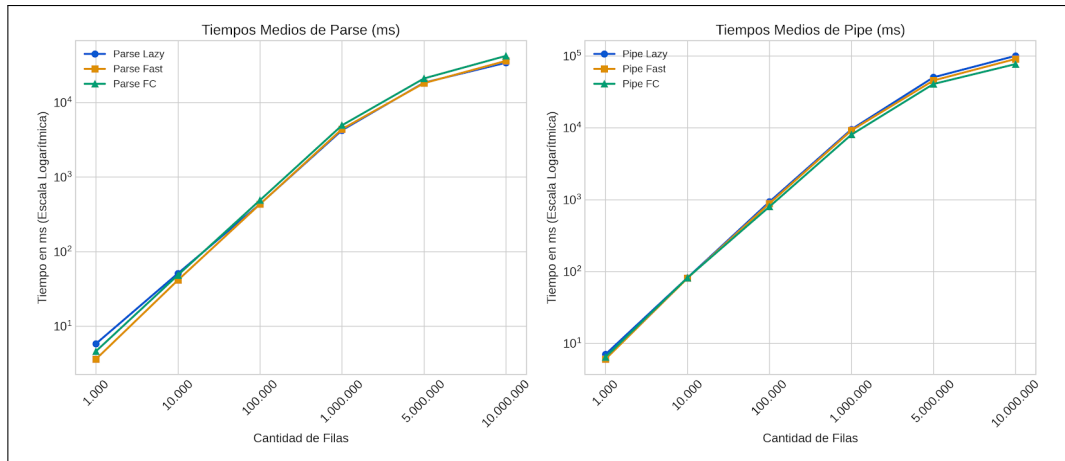


Figura 4: Comparación de las diferentes estrategias de parseo.

6. Descubrimiento del proceso mediante Alpha Miner

Una vez estructurados los eventos, el siguiente objetivo fue descubrir las relaciones existentes entre las diferentes actividades.

Para ello se implementó el algoritmo **Alpha Miner**.

El algoritmo analiza las trazas y estudia la relación entre pares de actividades. Para cada par de actividades A y B se comprueba si:

- A ocurre antes que B .
- B ocurre antes que A .
- Ambas situaciones ocurren.
- Ninguna de las dos situaciones ocurre.

A partir de estas observaciones se puede establecer una de las siguientes relaciones:

- $A \rightarrow B$: relación causal.
- $B \rightarrow A$: relación causal en sentido contrario.
- $A \leftrightarrow B$: relación bidireccional.
- $A \# B$: no existe relación observada.

El resultado de este análisis se almacena inicialmente en una **footprint matrix**, que permite representar las relaciones entre todas las actividades del registro.

	0	1	2	3	4	5	6	7	8	9
0. Recibir_Solicitud	#	->	#	#	#	#	#	#	#	#
1. Verificacion_Automatica	<-	#	->	->	#	#	#	#	#	#
2. Comprobar_Antecedentes	#	<-	#		->	#	#	->	#	#
3. Calculo_Scoring	#	<-		#	->	#	#	->	#	#
4. Requiere_Revision_Manual	#	#	<-	<-	#	->	#	#	#	#
5. Analisis_Riesgo_Manual	#	#	#	#	<-	#	->	#	#	->
6. Rechazo_Manual	#	#	#	#	#	<-	#	#	#	#
7. Aprobacion_Automatica	#	#	<-	<-	#	#	#	#	->	#
8. Emitir_Transferencia	#	#	#	#	#	#	#	<-	#	#
9. Aprobacion_Manual	#	#	#	#	#	<-	#	#	#	#

Figura 5: Ejemplo de matriz de relaciones.

A partir de esta matriz se genera un grafo del proceso, en el que los nodos representan actividades y las aristas representan las relaciones descubiertas.

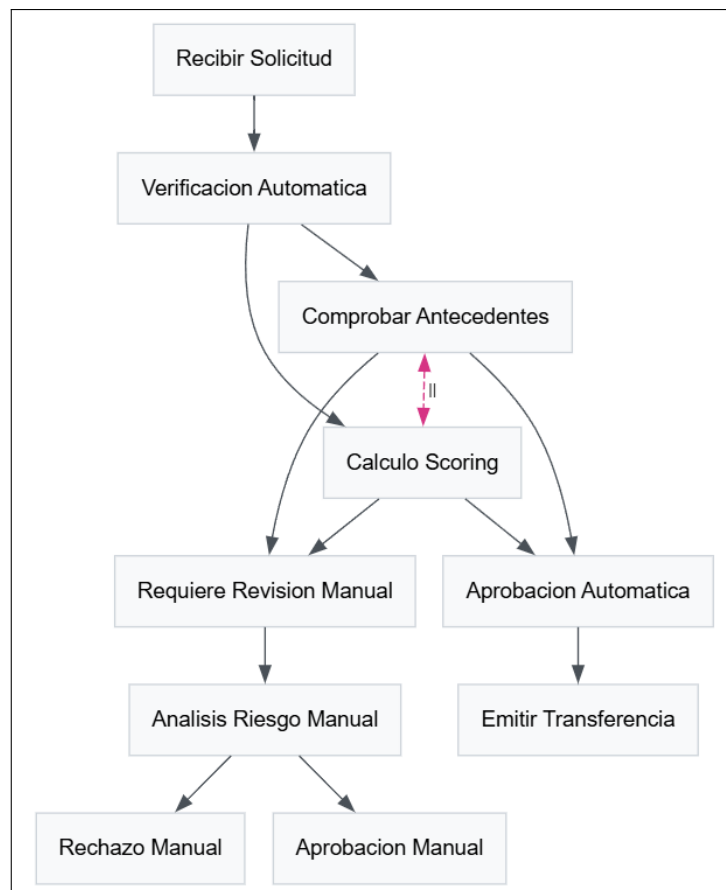


Figura 6: Grafo del proceso descubierto mediante Alpha Miner.

7. Tratamiento del ruido

La primera implementación del Alpha Miner presenta una limitación importante cuando se trabaja con datos reales: la presencia de **ruido**.

Si una relación $A \rightarrow B$ aparece en el 99% de los casos, pero existe un caso aislado en el que aparece $B \rightarrow A$, el análisis binario puede interpretar ambas relaciones como existentes y concluir que existe una relación bidireccional.

Esto puede provocar la aparición de conexiones que realmente no representan el comportamiento habitual del proceso.

El problema es especialmente relevante en registros grandes y con ejecuciones que contienen excepciones o comportamientos poco frecuentes.

Esta limitación motivó posteriormente la implementación del **Heuristic Miner**, que incorpora la frecuencia de las relaciones para realizar un análisis más robusto.

8. Performance Mining

Además de conocer la estructura del proceso, resulta interesante conocer cuánto tarda cada transición.

Para ello se implementó el módulo de **Performance Mining**.

El algoritmo recorre las trazas ordenadas cronológicamente y analiza los eventos consecutivos. Cuando encuentra una transición válida entre dos actividades $A \rightarrow B$, calcula el tiempo transcurrido entre ambas.

Los tiempos correspondientes a una misma transición se acumulan y posteriormente se calcula su media.

Si para una determinada transición existen n observaciones, el tiempo medio se calcula mediante:

$$\bar{t}(A, B) = \frac{1}{n} \sum_{i=1}^n t_i$$

donde t_i representa el tiempo transcurrido en la i -ésima aparición de la transición.

El resultado se almacena en una matriz de tiempos.

Esta información se incorpora posteriormente al grafo del proceso. Las aristas pueden mostrar el tiempo medio empleado en cada transición, permitiendo localizar las partes del proceso que presentan mayores tiempos de espera.

Además, se calcula el coste medio asociado a cada actividad. Si una actividad A aparece n veces con costes $c_1, \dots, c_n$, su coste medio es:

$$\bar{c}(A) = \frac{1}{n} \sum_{i=1}^n c_i$$

Las actividades cuyo coste supera un determinado umbral pueden destacarse visualmente en el grafo.

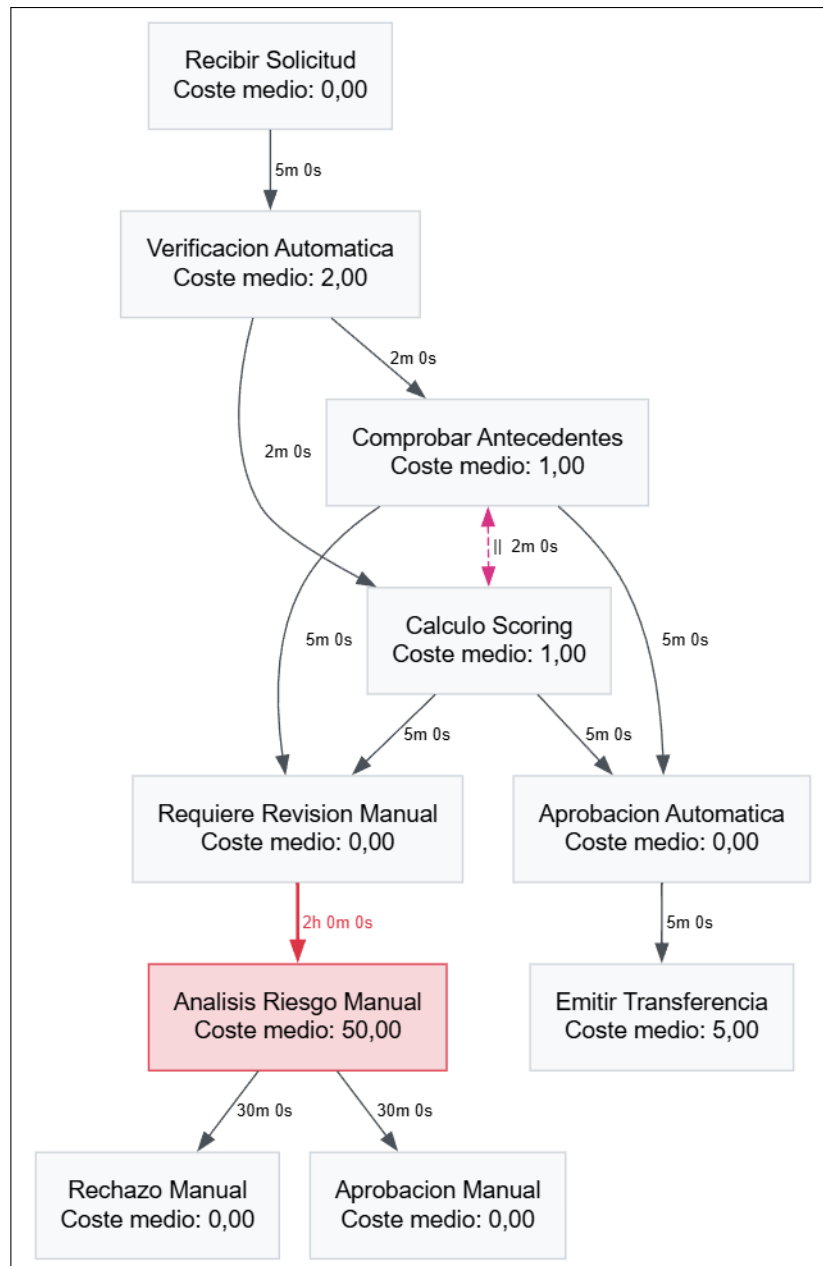


Figura 7: Visualización de la información de rendimiento.

9. Resource Mining

El **Resource Mining** permite analizar las relaciones entre los recursos que participan en el proceso.

Un recurso puede representar, por ejemplo, un sistema automático, una API, un empleado o cualquier otro participante registrado en el *event log*.

El algoritmo comienza identificando todos los recursos existentes mediante un conjunto de elementos únicos. Posteriormente recorre las trazas ordenadas y registra las transiciones entre recursos consecutivos.

Cada relación $A \rightarrow B$ incrementa la posición correspondiente de una matriz de relaciones.

Finalmente, esta información se representa mediante un grafo cuyos nodos corresponden a los recursos y cuyas aristas indican cuántas veces se produce una interacción direccional entre ellos.

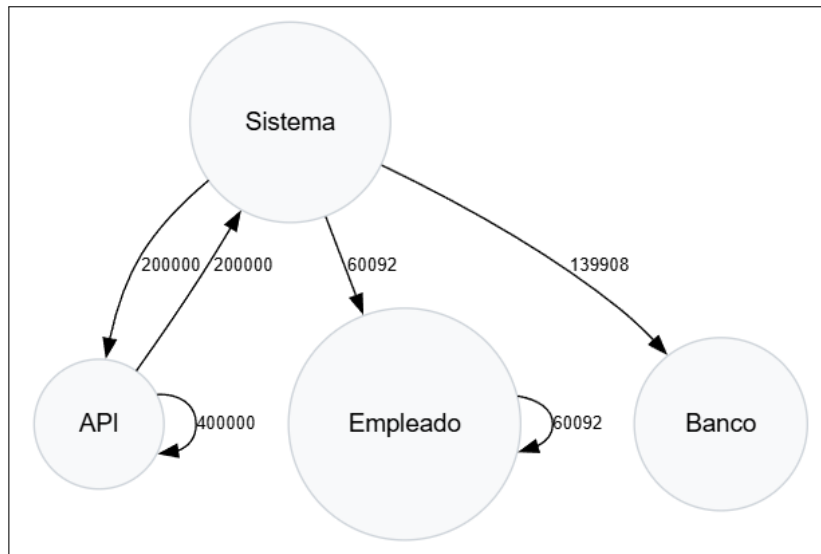


Figura 8: Grafo de relaciones entre recursos.

10. Heuristic Miner

Para solucionar las limitaciones del enfoque puramente binario utilizado inicialmente por Alpha Miner se desarrolló el **Heuristic Miner**.

En lugar de considerar únicamente si una relación aparece o no aparece, se analiza la frecuencia con la que aparece en cada dirección.

La dependencia entre dos actividades se calcula mediante:

$$Dependency(A, B) = \frac{|A >_L B| - |A <_L B|}{|A >_L B| + |A <_L B| + 1}$$

donde:

- $|A >_L B|$ representa el número de veces que A es seguida directamente por B .

- $|A <_L B|$ representa el número de veces que B es seguida directamente por A .

Un valor próximo a 1 indica una fuerte dependencia de A respecto a B , mientras que un valor próximo a -1 indica dependencia en el sentido contrario.

Valores próximos a cero indican que no existe una relación causal clara.

Para distinguir entre concurrencia y ausencia de relación se utiliza una segunda medida:

$$Dependency'(A, B) = \frac{|A >_L B| + |A <_L B|}{|A >_L B| + |A <_L B| + 1}$$

Esta medida permite determinar si una relación con dependencia cercana a cero puede corresponder realmente a actividades concurrentes.

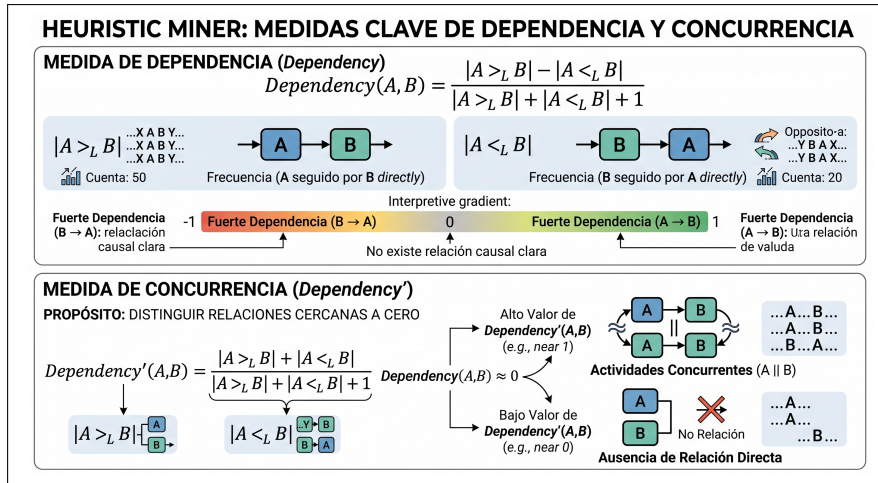


Figura 9: Medidas utilizadas por el Heuristic Miner.

10.1. Umbrales y filtrado del ruido

Durante las pruebas con registros pequeños se observó que la escasez de datos puede producir probabilidades poco representativas.

Por este motivo se introdujeron umbrales para determinar qué relaciones deben aparecer finalmente en el grafo.

En una primera prueba se utilizaron valores de:

$$Dependency = 0,45$$

y:

$$Dependency' = 0,70$$

Posteriormente se trabajó con registros superiores a 1,2 millones de líneas y se modificó la representación visual para utilizar un gradiente basado en la frecuencia de las relaciones.

Las relaciones más frecuentes se representan visualmente con mayor intensidad, mientras que las menos frecuentes aparecen con menor intensidad.

De esta forma es posible observar de manera inmediata cuál es el camino principal del proceso.

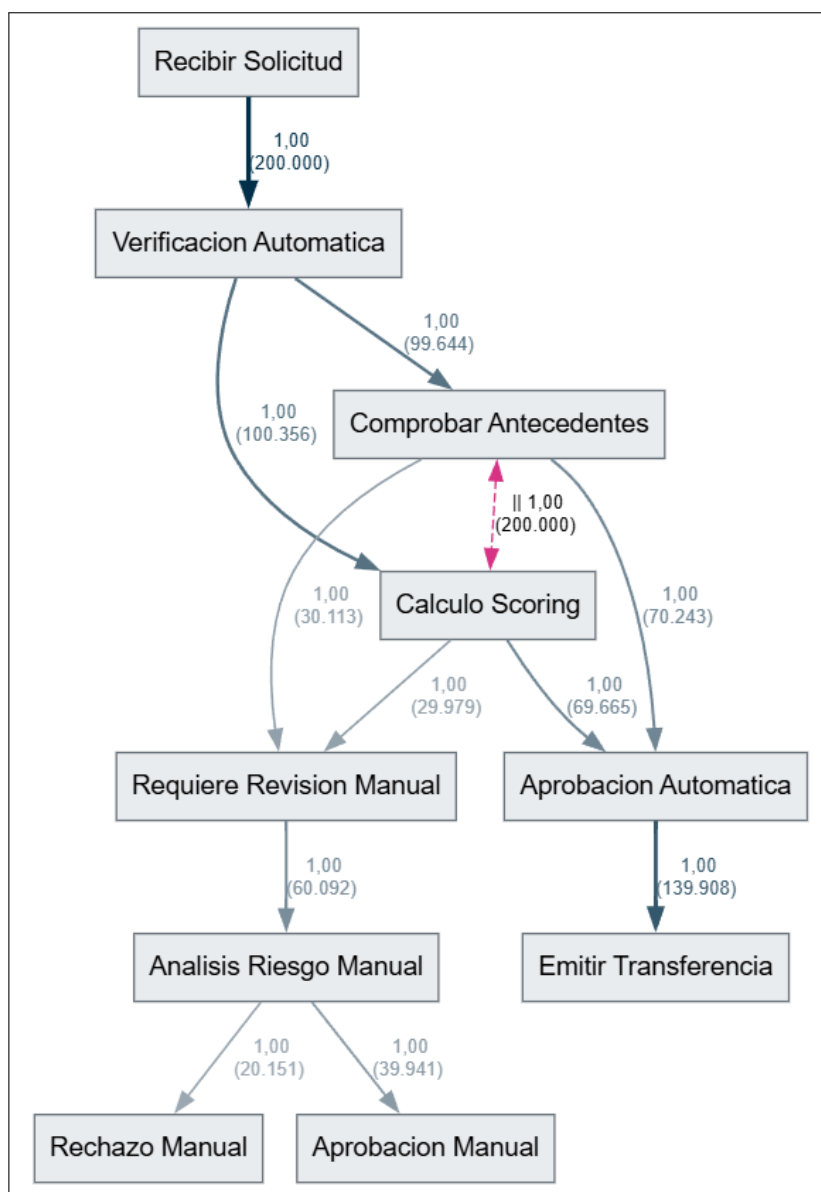


Figura 10: Heuristic Miner aplicado a un registro de gran tamaño.

Durante las pruebas finales también se detectó que determinadas relaciones podían aparecer únicamente una o dos veces. A pesar de su escasa frecuencia, estas apariciones podían provocar la creación de conexiones en el grafo.

Para evitar este comportamiento se introdujo un filtro basado en la frecuencia mínima de

las transiciones.

Inicialmente se estableció un umbral correspondiente al:

$$0,01 \%$$

del número total de líneas del registro.

Posteriormente este parámetro se hizo configurable desde la aplicación, permitiendo al usuario ajustar el nivel de filtrado según las características del conjunto de datos.

De esta forma, el usuario puede modificar los umbrales utilizados por el *MinerController* sin necesidad de modificar el código del algoritmo.

11. Análisis de variantes

No todos los casos de un proceso siguen necesariamente la misma secuencia de actividades.

Por ello se implementó el concepto de **Process Variant**, que representa una secuencia concreta de actividades.

Para cada variante se almacenan:

- Secuencia de actividades.
- Número de casos en los que aparece.
- Porcentaje sobre el total.
- Duración media.
- Coste medio.

El sistema permite obtener las n variantes más frecuentes del registro.

Por ejemplo, durante las pruebas realizadas se identificaron variantes automáticas que representaban aproximadamente el 35 % y el 34,8 % de los casos respectivamente.

También se identificaron variantes que requieren revisión manual y que presentaban costes y tiempos considerablemente superiores.

Este análisis permite conocer no solamente cuál es el proceso general, sino también qué caminos concretos siguen realmente los casos.


```
Top 5 Variants:
#1 (35,12% | 70.243 cases)
  Signature: Recibir_Solicitud->Verificacion_Automatizada->Calculo_Scoring->
    Comprobar_Antecedentes->Aprobacion_Automatizada->Emitir_Transferencia
  Mean Duration: 00:19:00 | Mean Cost: 9,00
#2 (34,83% | 69.665 cases)
  Signature: Recibir_Solicitud->Verificacion_Automatizada->Comprobar_Antecedentes->
    Calculo_Scoring->Aprobacion_Automatizada->Emitir_Transferencia
  Mean Duration: 00:19:00 | Mean Cost: 9,00
#3 (10,05% | 20.095 cases)
  Signature: Recibir_Solicitud->Verificacion_Automatizada->Calculo_Scoring->
    Comprobar_Antecedentes->Requiere_Revision_Manual->Analisis_Riesgo_Manual->
    Aprobacion_Manual
  Mean Duration: 02:44:00 | Mean Cost: 54,00
#4 (9,92% | 19.846 cases)
  Signature: Recibir_Solicitud->Verificacion_Automatizada->Comprobar_Antecedentes->
    Calculo_Scoring->Requiere_Revision_Manual->Analisis_Riesgo_Manual->
    Aprobacion_Manual
  Mean Duration: 02:44:00 | Mean Cost: 54,00
#5 (5,07% | 10.133 cases)
  Signature: Recibir_Solicitud->Verificacion_Automatizada->Comprobar_Antecedentes->
    Calculo_Scoring->Requiere_Revision_Manual->Analisis_Riesgo_Manual->Rechazo_Manual
  Mean Duration: 02:44:00 | Mean Cost: 54,00
```

Figura 11: Análisis de las principales variantes del proceso.

12. API Web

Una vez desarrollado y probado el núcleo de procesamiento se creó una API Web utilizando **ASP.NET Core**.

La API actúa como intermediaria entre la interfaz web y el motor de Minería de Procesos.

El cliente envía el archivo de eventos al servidor y el controlador ejecuta el procesamiento correspondiente.

Como resultado se devuelve un objeto JSON que contiene, entre otros elementos:

- Grafo heurístico.
- Grafo Alpha Miner.
- Grafo de recursos.
- Variantes principales.
- Lista de actividades.
- Matriz de dependencias.
- Matriz de concurrencia.

- Información de rendimiento.

Los grafos se generan en formato DOT para posteriormente ser interpretados y representados visualmente por el frontend.

13. Arquitectura Stateless

Una de las decisiones arquitectónicas importantes del proyecto es que la aplicación funciona de forma **stateless**.

Cuando el usuario carga un archivo, React lo envía a la API mediante `FormData`.

El servidor procesa temporalmente el archivo, construye las estructuras necesarias y genera los resultados.

Una vez devuelta la respuesta, el archivo temporal deja de ser necesario.

Los resultados tampoco se almacenan permanentemente en el servidor. React recibe el JSON y lo mantiene en el estado de la aplicación mediante `useState`.

Por tanto, si el usuario actualiza la página, los resultados desaparecen y es necesario volver a cargar el archivo.

Esta arquitectura presenta varias ventajas.

En primer lugar, evita mantener almacenados permanentemente los registros procesados. Esto resulta especialmente interesante porque los *event logs* pueden contener información potencialmente sensible, como identificadores de clientes o información financiera.

En segundo lugar, reduce las necesidades de almacenamiento del servidor, ya que los archivos cargados no se conservan permanentemente.

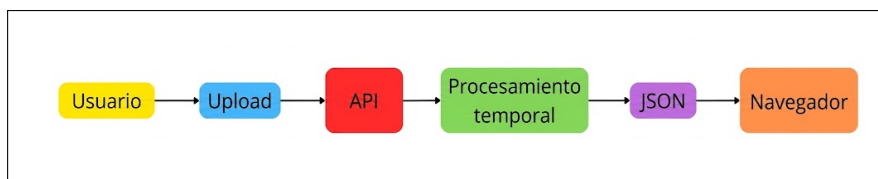


Figura 12: Flujo de procesamiento stateless.

14. Frontend

Una vez completado el backend se desarrolló la interfaz web utilizando **React y TailwindCSS**.

La interfaz sigue una estructura de tipo aplicación industrial, con un menú lateral desde el que el usuario puede acceder a las diferentes funcionalidades del sistema.

El archivo puede cargarse mediante:

- Arrastrar y soltar.
- Selección desde el explorador de archivos.

El frontend permite consultar los diferentes resultados generados por el backend y visualizar los grafos de forma interactiva.

También se incorporaron controles para modificar los parámetros utilizados por el Heuristic Miner.

De esta forma, el usuario puede ajustar los umbrales empleados para filtrar relaciones poco frecuentes sin modificar directamente la implementación.

La aplicación cuenta además con una pantalla principal que reúne la información más relevante del análisis y permite consultar los diferentes resultados desde una única interfaz.

Los grafos pueden descargarse en formato **SVG**, lo que permite conservar su calidad independientemente del tamaño al que se visualicen.



Figura 13: Pantalla principal de Process Miner.

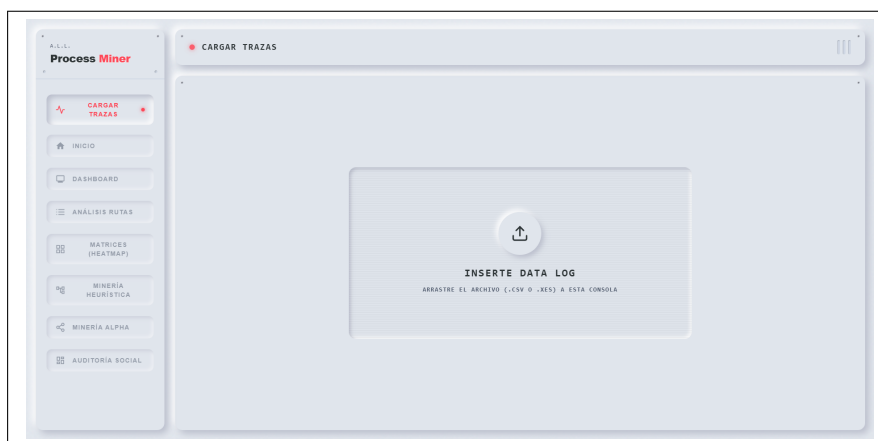


Figura 14: Carga de registros desde la interfaz web.

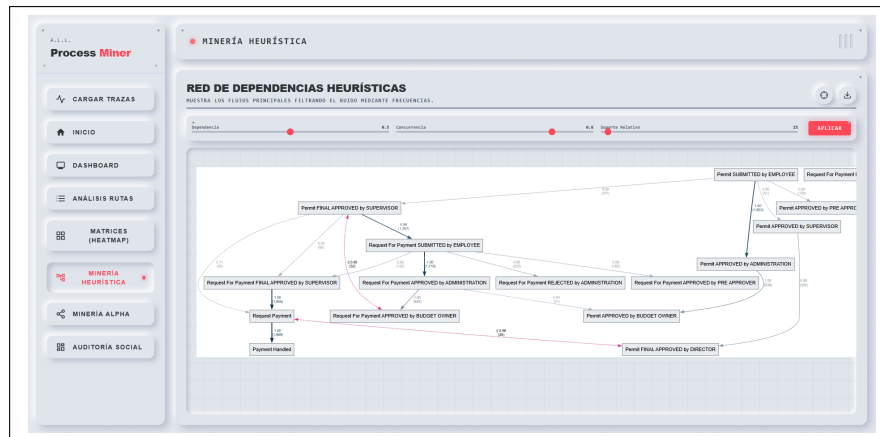


Figura 15: Visualización interactiva de un grafo.

15. Soporte de formatos

Inicialmente el sistema se desarrolló utilizando registros en formato CSV.

Durante las últimas etapas del proyecto se amplió el backend para permitir también la carga de archivos **XES**, un formato utilizado específicamente para representar registros de eventos en el ámbito de la Minería de Procesos.

De esta manera, la herramienta puede trabajar con dos formatos de entrada:

- CSV.
- XES.

La incorporación de XES permite utilizar directamente registros de eventos generados por herramientas y entornos habituales dentro del ámbito de la Minería de Procesos.

16. Pruebas de rendimiento y escalabilidad

La escalabilidad ha sido uno de los aspectos principales durante el desarrollo.

Se realizaron pruebas con registros que iban desde 1 000 hasta 10 000 000 de filas.

Los tiempos observados para el *pipeline* completo con la implementación finalmente seleccionada fueron aproximadamente:

Filas	Tiempo aproximado
1 000	0.008 s
10 000	0.08 s
100 000	0.8 s
1 000 000	8 s
10 000 000	72–81 s

Cuadro 1: Tiempos aproximados de procesamiento según el tamaño del registro.

A partir de estas pruebas se observó un comportamiento aproximadamente lineal para el *pipeline* utilizado, estimándose una complejidad:

$$O(n)$$

para el procesamiento del conjunto de datos bajo las condiciones de las pruebas realizadas.

Es importante destacar que estos resultados corresponden a las condiciones de las pruebas realizadas durante el desarrollo y que el rendimiento final depende del hardware y del entorno de ejecución.

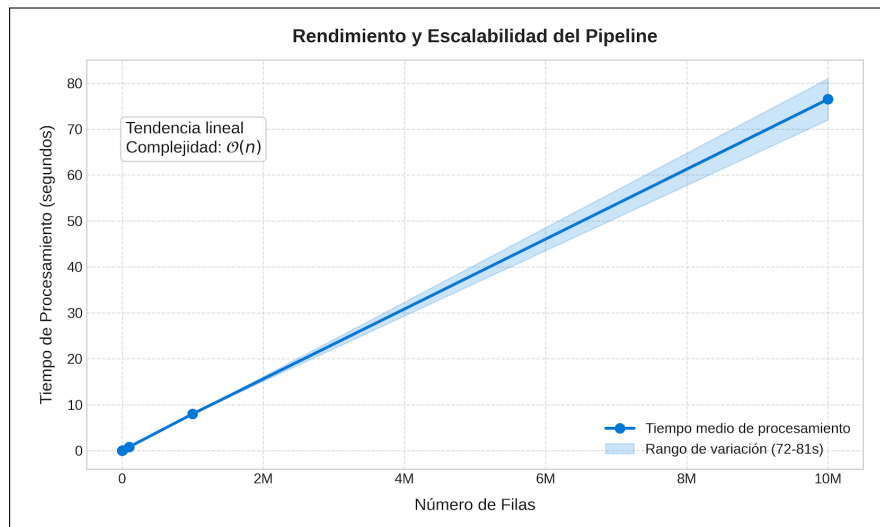


Figura 16: Escalabilidad del procesamiento en función del tamaño del registro.

17. Despliegue

Para convertir el proyecto en una aplicación accesible desde Internet se desplegó el backend utilizando **Render** y el frontend mediante **Vercel**.

Para facilitar el despliegue del backend se creó un **Dockerfile** con las dependencias necesarias.

Durante las pruebas de despliegue apareció un problema de consumo de memoria al intentar procesar directamente el archivo de estrés.

El problema se solucionó modificando el procesamiento de las trazas en el `MinerController`, pasando de trabajar con listas completas a utilizar `IEnumerable`.

Este cambio permite procesar los datos de manera más eficiente y redujo significativamente el consumo de memoria, permitiendo procesar correctamente el archivo de estrés en el entorno de despliegue.

Además, debido a que el plan gratuito utilizado para Render puede apagar la API después de un periodo de inactividad, se desarrolló una secuencia de ignición que realiza peticiones a la API hasta comprobar que se encuentra disponible y, posteriormente, permite continuar con la subida del archivo.

Finalmente, el frontend fue desplegado en Vercel, incorporando los metadatos e iconos necesarios para completar la aplicación.

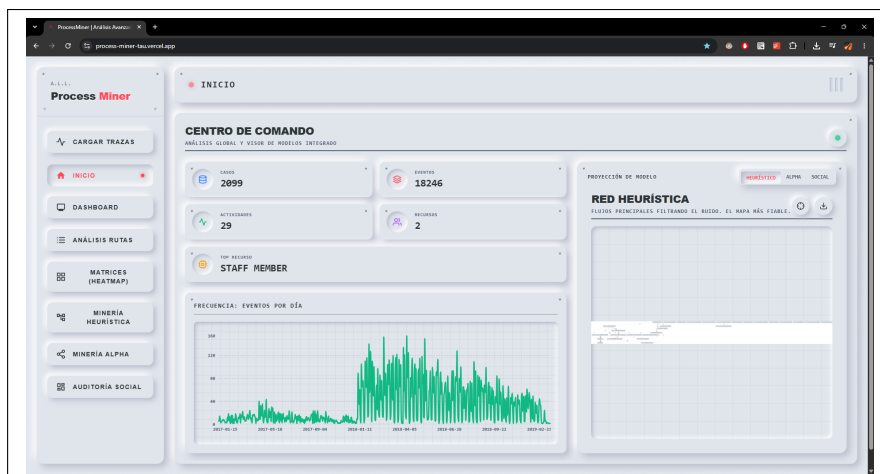


Figura 17: Aplicación desplegada en producción.

18. Flujo completo de funcionamiento

El funcionamiento completo de la herramienta puede resumirse mediante las siguientes etapas.

18.1. Carga del registro

El usuario selecciona o arrastra un archivo CSV o XES desde la interfaz web.

18.2. Envío a la API

React envía el archivo al backend mediante una petición HTTP.

18.3. Parseo

La API procesa el registro utilizando el parser optimizado.

18.4. Construcción de las trazas

Los eventos se agrupan por caso y se ordenan cronológicamente.

18.5. Descubrimiento del proceso

Se ejecutan los algoritmos de descubrimiento correspondientes:

- Alpha Miner.
- Heuristic Miner.

18.6. Análisis adicional

Se calculan:

- Tiempos de transición.
- Costes medios.
- Relaciones entre recursos.
- Variantes del proceso.
- Dependencias.
- Concurrencias.

18.7. Generación de resultados

Los resultados se transforman en estructuras JSON y grafos DOT.

18.8. Visualización

React recibe los resultados y los muestra mediante las diferentes vistas de la aplicación.

18.9. Exportación

Los grafos pueden descargarse en formato SVG.

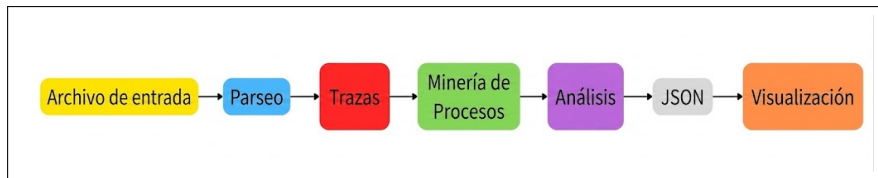


Figura 18: Flujo completo de funcionamiento de Process Miner.

19. Resultados obtenidos

El resultado final es una herramienta capaz de transformar un registro de eventos en una representación estructurada del proceso y proporcionar información adicional sobre su comportamiento.

Entre las principales funcionalidades desarrolladas se encuentran:

- Lectura de registros CSV.
- Lectura de registros XES.
- Agrupación de eventos en trazas.
- Ordenación temporal de eventos.
- Parser optimizado mediante `ReadOnlySpan<char>`.
- Sistema de caché para reducir la creación de cadenas.
- Alpha Miner.
- Footprint Matrix.
- Heuristic Miner.
- Análisis de dependencia.
- Análisis de concurrencia.
- Filtrado de relaciones poco frecuentes.
- Performance Mining.
- Análisis de costes.
- Resource Mining.
- Análisis de variantes.
- Generación de grafos.
- API REST mediante ASP.NET Core.

- Interfaz web mediante React y TailwindCSS.
- Carga mediante drag & drop.
- Configuración de umbrales.
- Exportación de grafos en SVG.
- Despliegue del backend y frontend.

La API consigue reunir los diferentes resultados en una única respuesta, permitiendo que el frontend pueda representar de manera conjunta el modelo del proceso, la información de rendimiento, las relaciones entre recursos y las variantes más frecuentes.

La combinación de estas funcionalidades permite pasar de un registro de eventos compuesto por un gran número de filas a una representación visual del proceso acompañada de información cuantitativa.

20. Conclusiones

El desarrollo de Process Miner ha permitido construir una herramienta completa para el procesamiento y análisis de registros de eventos mediante técnicas de Minería de Procesos.

El proyecto comenzó con una implementación básica capaz de leer un archivo y construir trazas y evolucionó progresivamente hasta convertirse en una aplicación web completa con diferentes algoritmos de análisis.

Uno de los aspectos más importantes ha sido el tratamiento de la eficiencia. Los experimentos realizados con registros de hasta diez millones de filas permitieron identificar los costes asociados a las diferentes estrategias de parseo y llevaron a seleccionar una implementación optimizada basada en `ReadOnlySpan<char>` y caché.

Los experimentos mostraron que la optimización debe evaluarse teniendo en cuenta el *pipeline* completo y no únicamente una etapa individual. Una técnica que incrementa ligeramente el tiempo de parseo puede resultar beneficiosa cuando se consideran todas las fases posteriores del procesamiento.

También se ha comprobado la importancia de tener en cuenta el ruido de los registros. La implementación inicial de Alpha Miner resulta especialmente sensible a relaciones poco frecuentes, lo que motivó el desarrollo del Heuristic Miner y la introducción de medidas de dependencia, concurrencia y filtrado por frecuencia.

El sistema no se limita al descubrimiento de la estructura del proceso. El Performance Mining permite estudiar los tiempos de transición y los costes de las actividades, mientras

que el Resource Mining permite observar las interacciones entre los participantes del proceso.

Por otro lado, el análisis de variantes permite identificar los caminos más frecuentes seguidos por los casos y estudiar sus diferencias en términos de frecuencia, duración y coste.

La incorporación de los formatos CSV y XES amplía los tipos de registros que pueden utilizarse como entrada y hace que la herramienta sea más flexible para distintos conjuntos de datos.

Finalmente, la incorporación de una API web y una interfaz desarrollada con React permite utilizar todas estas funcionalidades mediante una aplicación accesible desde el navegador.

El diseño *stateless* evita la necesidad de almacenar permanentemente los registros procesados y simplifica la arquitectura del sistema. Además, la utilización de `IEnumerable` en el procesamiento final permitió reducir el consumo de memoria durante el despliegue y procesar registros de gran tamaño en un entorno con recursos limitados.

En conjunto, el proyecto demuestra cómo un registro de eventos, inicialmente compuesto por grandes cantidades de datos aparentemente inconexos, puede transformarse mediante técnicas algorítmicas en una representación visual y cuantitativa del proceso real.

La herramienta desarrollada integra en una única aplicación las diferentes fases necesarias para realizar este análisis: desde la carga y transformación eficiente de los datos hasta el descubrimiento del proceso, el análisis de rendimiento y recursos, el estudio de variantes y la visualización interactiva de los resultados.